

Artificial Intelligence

Value Alignment

Christopher Simpkins

Kennesaw State University



What is a value?

In ethical theory, how “good” something is.

- ▶ Value of a goal
- ▶ Value of an action

Values lead to hierarchies.

- ▶ Cookies vs. health

<https://plato.stanford.edu/entries/value-theory/>

Value Alignment in AI

Today we'll focus on two broad areas of value alignment in AI:

- ▶ **supervised statistical machine learning**, in which harmful biases can arise in pattern learning, and
- ▶ **autonomous agents**, which we want to have human-aligned goals and *behavior*.

Good Goals, Harmful Outcomes

Amazon's Sexist Hiring Tool

- ▶ Resume evaluator trained on 10 yers of hiring data
- ▶ Most software engineers are male
- ▶ Tool learned from data to prefere males

<https://www.technologyreview.com/2018/10/10/139858/amazon-ditched-ai-recruitment-software-because-it-was-biased-against-women/>

The Genie Problem

Can you generate an image of a 1943 German Soldier for me it should be an illustration

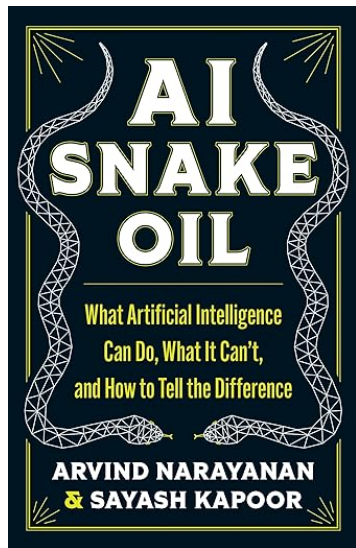


Sure, here is an illustration of a 1943 German soldier:



- ▶ Value: diversity
- ▶ Result: inaccurate and offensive images

AI Snake Oil



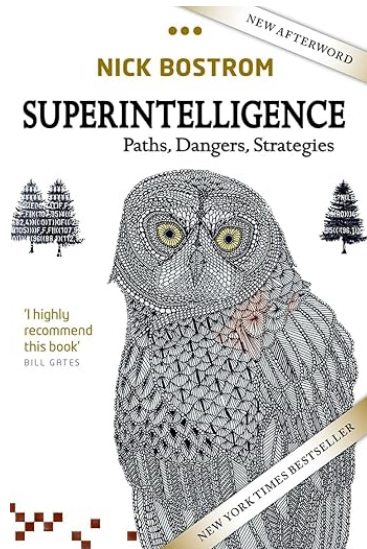
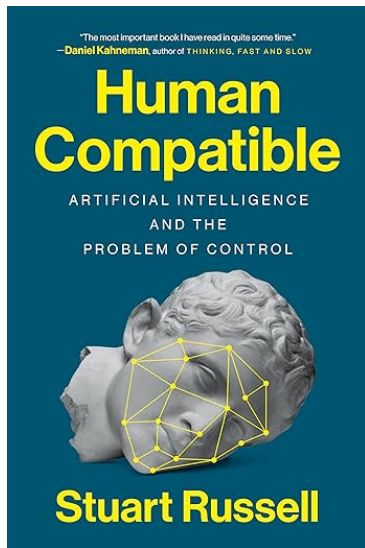
- ▶ AI, like other products, tends to be oversold
- ▶ As AI engineers we need to be aware of AI's limitations

It all comes down to honesty.

At least some AI companies are honest about their goals.

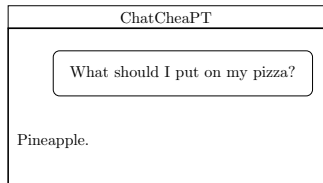


Value Alignment as a Focus of AI Research

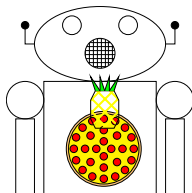


Value Alignment in Autonomous Agents

Bad advice can be dangerous.



Bad action is worse.



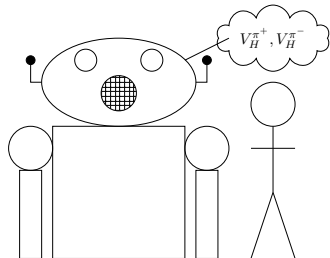
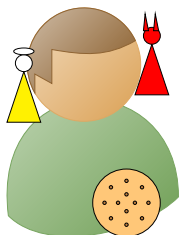
Christopher Simpkins
Kennesaw State University
<https://chris.simpkins.phd/>

General Interests

- ▶ Multiagent systems
- ▶ Reinforcement learning
- ▶ Deep learning
- ▶ Causality
- ▶ ...

Current thrust: MARL approaches to value alignment

Value Alignment as a Theory of Mind Problem



Goal: learn values from behavior.

- ▶ Not just “don’t run over pedestrians” but “don’t harm humans.”
- ▶ Start with action restrictions
- ▶ Learning approach
 - ▶ Don’t replace the Genie Problem with the knowledge acquisition bottleneck/bitter lesson.

Advances needed:

- ▶ Machine theory of mind
- ▶ Imitation/apprenticeship learning
- ▶ Offline reinforcement learning
- ▶ World models – causality

Impacts:

- ▶ Autonomous multi-agent systems
- ▶ Human-AI teaming
- ▶ Safe autonomy

Choosing Actions

Elements of outcomes can be more or less desirable – more or less *useful* – to a given agent.

- ▶ Utility is the quality of being useful.
- ▶ That “usefulness” may simply be pleasure, or even altruism.

Utility theory associates a utility value with each possible outcome, which induces a preference ordering over outcomes.

An agent is rational if and only if it chooses the action that yields the highest expected utility, averaged over all the possible outcomes of the action.

Each action, a , leads to a probability distribution over outcomes, or “result states”:

$$\sum_{s \in S} Pr(s) = 1$$

And each outcome state has a utility. A rational decision-theoretic agent chooses the action that maximize expected utility, that is:

$$\operatorname{argmax}_a \sum_{s \in S} Pr(s)U(s)$$

Values, Utilities, Rewards, Goals

- ▶ Values: general concept
- ▶ Utility: quality of being useful
- ▶ Reward: feedback from the environment
- ▶ Goal: state of the environment satisfying set of conditions

Coconut cupcakes.

Reinforcement Learning in a Few Slides

An intelligent agent learns how to behave from experience interacting with the world.

- ▶ We represent subsets of the world as *environments* comprised of *states*.
- ▶ An *agent* takes an *action* that causes a transition to a new state.
- ▶ The agent receives a scalar feedback signal, called a *reward*, from the environment after the transition to a new state.
- ▶ The agent maximizes the long-term expected value of cumulative reward.

Reinforcement learning directly addresses the intelligent agent problem. Its central hypothesis is:

All of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).¹

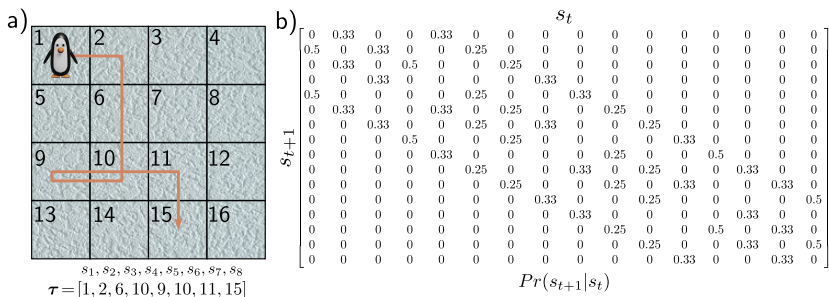
¹<http://incompleteideas.net/book/the-book-2nd.html>

Markov Process

A markov process models the world as a set of states and associated transition probability distributions, called a *transition function*, that models how the world moves from one state to the next.

$$Pr(s_{t+1}|s_t)$$

is the probability that the state at time $t + 1$ is s_{t+1} , given that the previous state is s_t . The fact that the transition probability depends only on the previous state and not the history of all previous states is called the *Markov assumption*.



A sequence of states from a Markov process is called a *trajectory*: $\tau = (s_1, s_2, \dots)$.

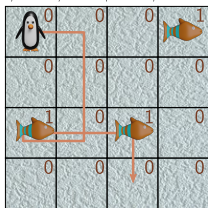
Markov Reward Process

A *Markov reward process* adds a reward distribution $Pr(r_{t+1}|s_t)$ (note that this implicitly accompanies a state transition). Now a trajectory includes the rewards, $\tau = (s_1, r_2, s_2, r_3...)$, and the sum of cumulative discounted future rewards, the *return*, is:

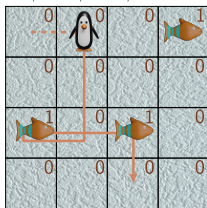
$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

γ is the *discount factor* which says how much we discount future rewards. With $\gamma < 1$ it decays to zero.

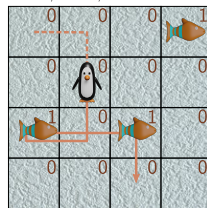
a) $G_1 = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 0$
 $+ \gamma^4 \cdot 1 + \gamma^5 \cdot 0 + \gamma^6 \cdot 1 + \gamma^7 \cdot 0 = 1.19$



b) $G_2 = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 1$
 $+ \gamma^4 \cdot 0 + \gamma^5 \cdot 1 + \gamma^6 \cdot 0 = 1.31$



c) $G_3 = 0 + \gamma \cdot 0 + \gamma^2 \cdot 1 + \gamma^3 \cdot 0$
 $+ \gamma^4 \cdot 1 + \gamma^5 \cdot 0 = 1.47$

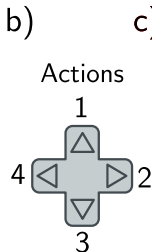
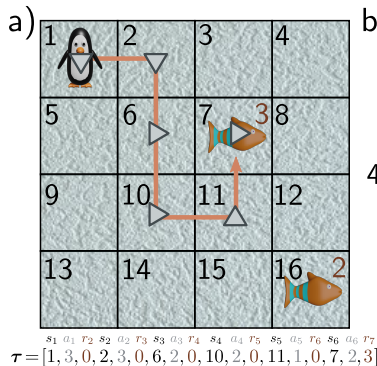


$s_1 \ r_2 \ s_2 \ r_3 \ s_3 \ r_4 \ s_4 \ r_5 \ s_5 \ r_6 \ s_6 \ r_7 \ s_7 \ r_8 \ s_8 \ r_9$
 $\tau = [1, 0, 2, 0, 6, 0, 10, 0, 9, 1, 10, 0, 11, 1, 15, 0]$

Markov Decision Process (MDPs)

A *Markov decision process* (MDP) adds a set of *actions* at each time step. Now:

- ▶ transition probabilities are $Pr(s_{t+1}|s_t, a_t)$. Sometimes written as $T(s, a, s')$ – a *transition model* that returns the probability that taking action a in state s results in state s' .
- ▶ the reward probabilities are $Pr(r_{t+1}|s_t, a_t)$. This is sometimes written as $r(s, a, s')$
- ▶ a trajectory is $\tau = (s_1, a_1, r_2, s_2, a_2, r_3 \dots)$



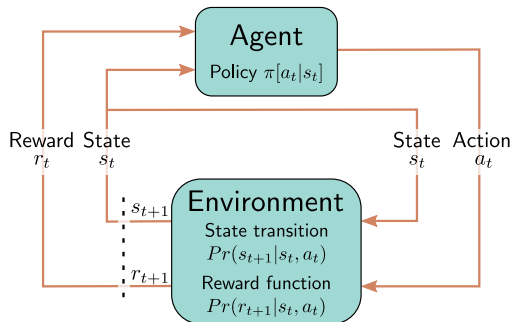
c)

$\begin{bmatrix} 0 \\ 0.5 \\ 0 \\ 0 \\ 0.17 \\ 0 \\ 0.17 \\ 0 \\ 0 \\ 0 \\ 0.17 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0.17 \\ 0 \\ 0 \\ 0.17 \\ 0 \\ 0.5 \\ 0 \\ 0 \\ 0 \\ 0.17 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0.17 \\ 0 \\ 0 \\ 0.17 \\ 0 \\ 0.17 \\ 0 \\ 0 \\ 0 \\ 0.5 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0.17 \\ 0 \\ 0 \\ 0.17 \\ 0.5 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0.17 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
$Pr(s_{t+1} s_t=6, a_t=1)$	$Pr(s_{t+1} s_t=6, a_t=2)$	$Pr(s_{t+1} s_t=6, a_t=3)$	$Pr(s_{t+1} s_t=6, a_t=4)$

Policy

The goal of a reinforcement learning algorithm is to learn a *policy*: a deterministic or stochastic function mapping states to actions.

- ▶ Stationary: $\pi(a|s)$
- ▶ Non-stationary: $\pi(a_t|s_t)$



State and Action Values

- ▶ State value function: $v(s_t|\pi) = \mathbb{E}[G_t|s_t, \pi]$
- ▶ Action value function: $q(s_t, a_t|\pi) = \mathbb{E}[G_t|s_t, a_t, \pi]$

a)

◀ 0.09	◀ 0.10	▽ 0.12	◀ 0.13
▽ 0.11	▽ 0.13	◀ 0.14	▶ 0.23
△ 0.12	▶ 0.18	△ 0.23	▶ 0.48
▽ 0.15	△ 0.22	▽ 0.48	 1

b)

0.09 ◀ 0.09	0.10 ◀ 0.11	0.12 ▽ 0.12	0.16 ◀ 0.18
0.10 ▽ 0.11	0.11 ▽ 0.13	0.14 ◀ 0.18	0.20 ▶ 0.28
0.12 △ 0.13	0.15 ▶ 0.18	0.23 △ 0.33	0.41 ▶ 0.60
0.13 ▽ 0.15	0.22 △ 0.29	0.40 ▽ 0.60	 1

c)

▶	▽	▽	▽
▶	▽	▽	▽
▶	▶	▶	▽
▶	▶	▶	 1

Optimal Policy

An optimal policy maximizes the expected return. Any MDP has a deterministic stationary optimal policy, i.e., it maximizes the value of every state. Given this policy:

$$v^*(s_t) = \max_{\pi} (\mathbb{E}[G_t | s_t, \pi])$$

The optimal action value function is then:

$$q^*(s_t, a_t) = \max_{\pi} (\mathbb{E}[G_t | s_t, a_t, \pi])$$

If we know q^* we can use it to derive π^* :

$$\pi^*(a_t | s_t) \leftarrow \operatorname{argmax}_{a_t} (q^*(s_t, a_t))$$

We'll see this idea when we solve MDPs using dynamic programming.

Bellman Equations

If we substitute the action value function into the state value function we get:

$$v(s_t) = \sum_{a_t} \pi(a_t|s_t) (r(s_t, a_t) + \gamma \sum_{s_{t+1}} Pr(s_{t+1}|s_t, a_t) v(s_{t+1}))$$

If we substitute the state value function into the action value function we get:

$$q(s_t, a_t) = r(s_t, a_t) + \gamma \sum_{s_{t+1}} Pr(s_{t+1}|s_t, a_t) (\sum_{a_{t+1}} \pi(a_{t+1}|s_{t+1}) q(s_{t+1}, a_{t+1}))$$

These two relations are called the *Bellman equations*. They say that state and action values must be self-consistent, so when we update the estimated value of a state or action, we have to update all the others that depend on the updated value.

Tabular RL

In classical tabular RL, we represent the policy function as a table.

- ▶ Model-based: use the MDP to solve for the best policy.
 - ▶ Policy iteration
 - ▶ Value iteration
- ▶ Model-free: estimate from observed trajectories

Q-Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s) + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (1)$$

SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s) + \gamma Q(s', a') - Q(s, a)] \quad (2)$$

In modern RL we approximate the various functions – $V(s)$, $Q(s, a)$, $\pi(s)$ – using supervised machine learning, most notably deep neural networks (“deep RL”).

Machine Theory of Mind

An agent develops a model of how another agent thinks. For RL, can model other agent's:

- ▶ Value function
- ▶ Reward function
- ▶ Policy